

Internship Reflection

Penetration Testing Engagements

Christian Bai | RISCPoint Advisory Group | Old Dominion University

Penetration Testing Internship Reflection

My penetration testing internship provided hands-on experience in offensive security operations. Working under established Rules of Engagement with multiple clients, I developed practical skills in vulnerability identification, exploitation techniques, and professional security documentation. Two projects stood out the past few weeks. The first was my encounter with AI system penetration testing, where I successfully extracted a complete system prompt through direct prompt injection. The second was a file upload vulnerability assessment where I bypassed client-side validation by intercepting the upload request and modifying the file extension.

Throughout the internship, Burp Suite Pro has become my primary platform for web application security testing. The tool's intercepting proxy functionality allowed me to analyze and manipulate HTTP requests in ways that revealed security flaws across multiple client applications. I learned to systematically test for OWASP Top 10 vulnerabilities to perform detailed manual validation of potential findings. One significant learning experiences involved understanding the difference between scanner-identified issues and actual exploitable vulnerabilities. Early in the internship, I would often accept Burp Scanner results at face value, only to discover during manual testing that many flagged issues were either false positives or properly implemented security controls. For example, I encountered several instances where what appeared to be SQL injection vulnerabilities were actually Web Application Firewall detection responses. This taught me the critical importance of manual verification and understanding the context of application behavior before classifying findings.

The most unexpected aspect of my internship was encountering AI system security testing for the first time. When assigned to assess an application that integrated a large language model chatbot interface, I initially approached it using traditional web application testing methodologies. However, I quickly realized that AI systems present entirely novel attack surfaces that require fundamentally different testing approaches.

My breakthrough came when researching prompt injection techniques. Unlike SQL injection or cross-site scripting, which exploit how applications process data, prompt injection targets the AI

model's instruction-following behavior itself. The concept that an AI system's core instructions could be extracted through carefully crafted user input was initially counterintuitive to me.

After studying documented prompt injection techniques and experimenting with various approaches, I successfully obtained the complete system prompt from the target AI application through direct prompt injection. The technique involved crafting inputs that convinced the model to output its initialization instructions, essentially revealing how the AI was configured to behave. The extracted prompt contained detailed instructions about the AI's role, capabilities, restrictions, and behavioral guidelines. This information could be leveraged to circumvent intended limitations or craft more sophisticated attacks.

This finding had significant implications for the client. The exposed system prompt revealed specific restrictions the AI was designed to enforce, content filtering approaches, and behavioral boundaries that could be exploited by malicious users. I documented this as a High severity finding, noting that system prompt exposure enables attackers to understand and potentially bypass security controls implemented at the AI instruction level.

The experience changed my understanding of security testing. AI systems require evaluating the robustness of natural language instructions and the model's resistance to manipulation through conversational techniques.

Another project involved assessing a client application with file upload functionality. File upload features are dangerous when improperly secured, as they can allow attackers to introduce malicious code into application environments. My testing methodology focused on identifying weaknesses in how the application validated, stored, and processed uploaded files.

Initial reconnaissance revealed that the application accepted various image types for upload. I began by testing the application's validation mechanisms, attempting to upload files with executable extensions. The application initially rejected these based on extension filtering. However, through systematic testing in Burp Suite, I discovered that the validation was client-side only and could be bypassed by intercepting the upload request and modifying the file extension after validation occurred. Unfortunately, time constraints stopped me before I was able to leverage this foothold into a more severe finding like stored XSS or RCE, however it still represented a valuable data point for the client. The application's failures stemmed from only a few items. It relied on client-side validation and lacked proper access controls on uploaded content. The vulnerability's existence demonstrates why security cannot rely on single control points but must implement multiple layers of protection.

I would say that these experiences enhanced my technical capabilities and professional understanding of penetration testing. The internship reinforced that cybersecurity is a rapidly evolving field

requiring continuous learning. The emergence of AI systems as a new attack surface exemplifies how security professionals must constantly adapt to new technologies and threat models. My successful extraction of an AI system prompt represents not just a technical achievement but also an early step in developing expertise in what will likely become a critical domain of security practice. As I move forward in my career, I carry forward a deeper understanding of security principles and the importance of ethical practice within established authorization frameworks.